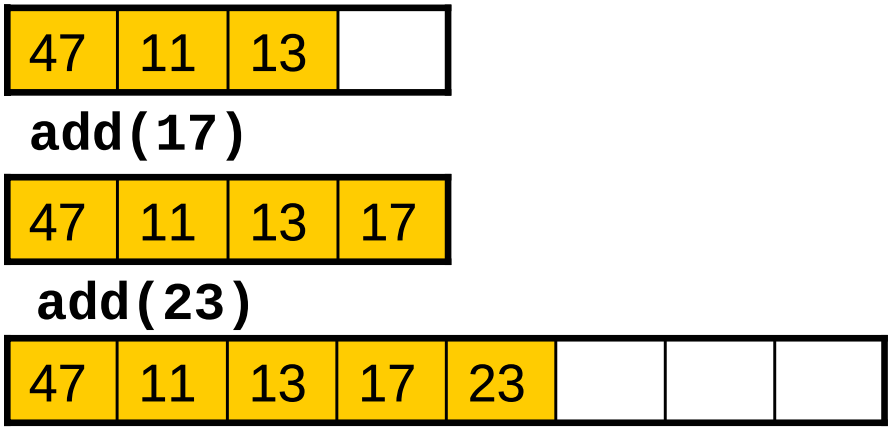
Ziele dieser Semesterwoche 2

- Sie können folgende Datenstrukturen implementieren:
  - *Listen*
  - *Stacks*
  - *Queues*
- Sie verstehen Iteratoren und können diese anwenden.

Array-List

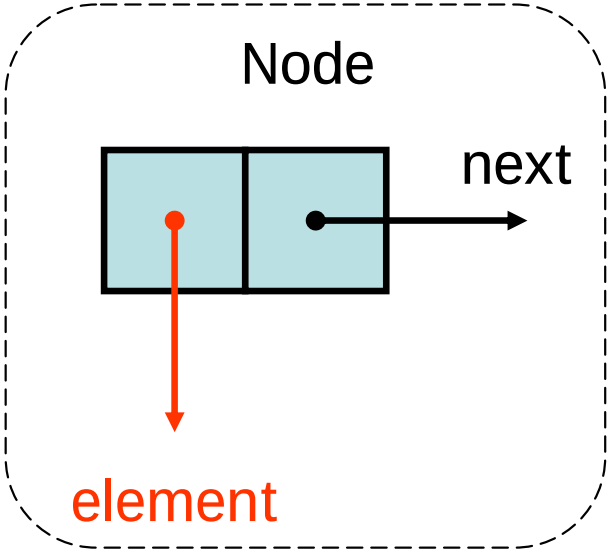
Eine *Array-List* kapselt einen Array, welcher sich in der Grösse automatisch anpasst:



- Der interne Array vergrössert sich automatisch wenn er voll ist (in diesem auf das Doppelte).
- Er verkleinert sich auch wieder automatisch wenn Zellen gelöscht werden.
- `add(17)` benötigt nur *O(1)* Laufzeit,
  wo hingegen `add(23)` wegen dem Umkopieren dann *O(n) !*

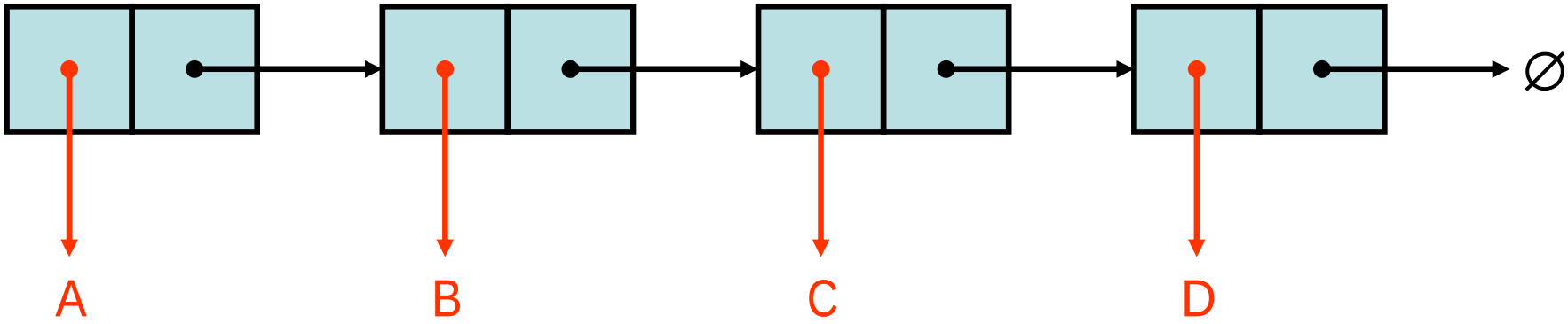
Linked-Lists: Einfach verkettete Liste

Eine einfach verkettete Liste besteht aus einer Sequenz von Knoten.



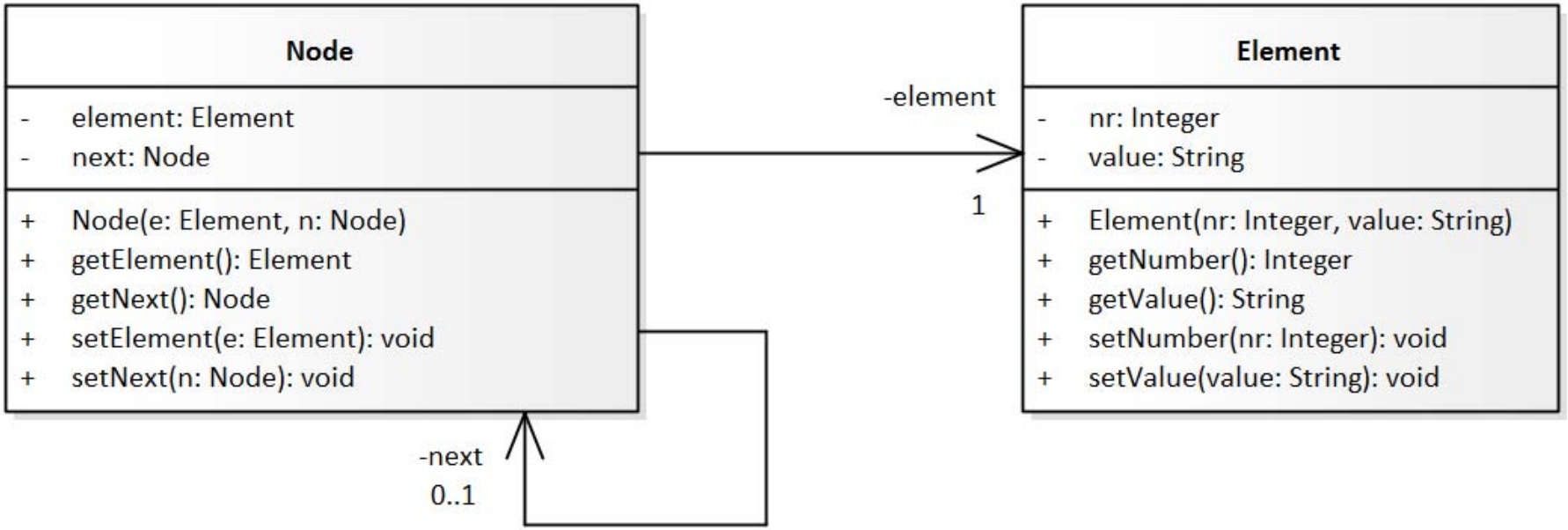
Jeder ***Knoten (Node)*** besitzt

- Ein ***Element*** (den Inhalt)
- Einen ***Link*** zum nächsten Knoten



Einfach verkettete Liste

Die Klassen *Node* + *Element* :



Java:

```
public class Node {
  private Element element;
  private Node next;

  public Node(Element e, Node n) {
    element = e;
    next = n;
  }
...
```

```
public class Element {
  private int nr;
  private String value;

...
```

Python:

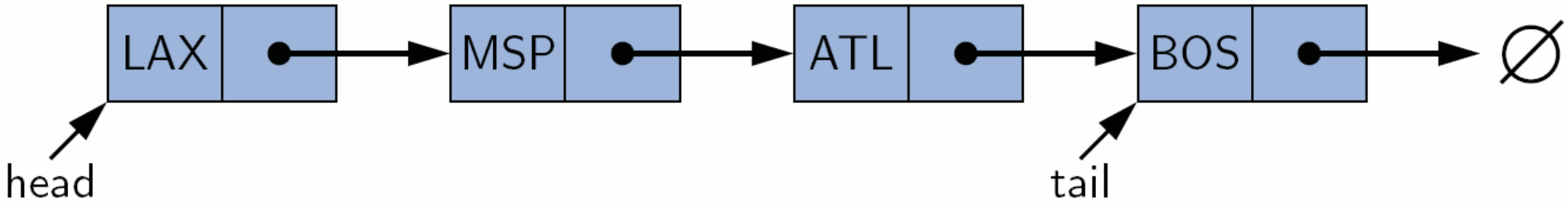
```
class Node:

  def __init__(self, e, n):
    self._element = e
    self._next = n

 ...
```

```
class Element:

  def __init__(self, nr, value):
    self._nr = nr
    self._value = value

 ...
```

Einfach verkettete Liste (Singly-Linked-List)

Eine ***einfach verkette Liste (Singly-Linked-List)*** besteht aus einer Sammlung von Knoten.

Die Reihenfolge ist bestimmt durch "Nachfolge" / "nächster Knoten".

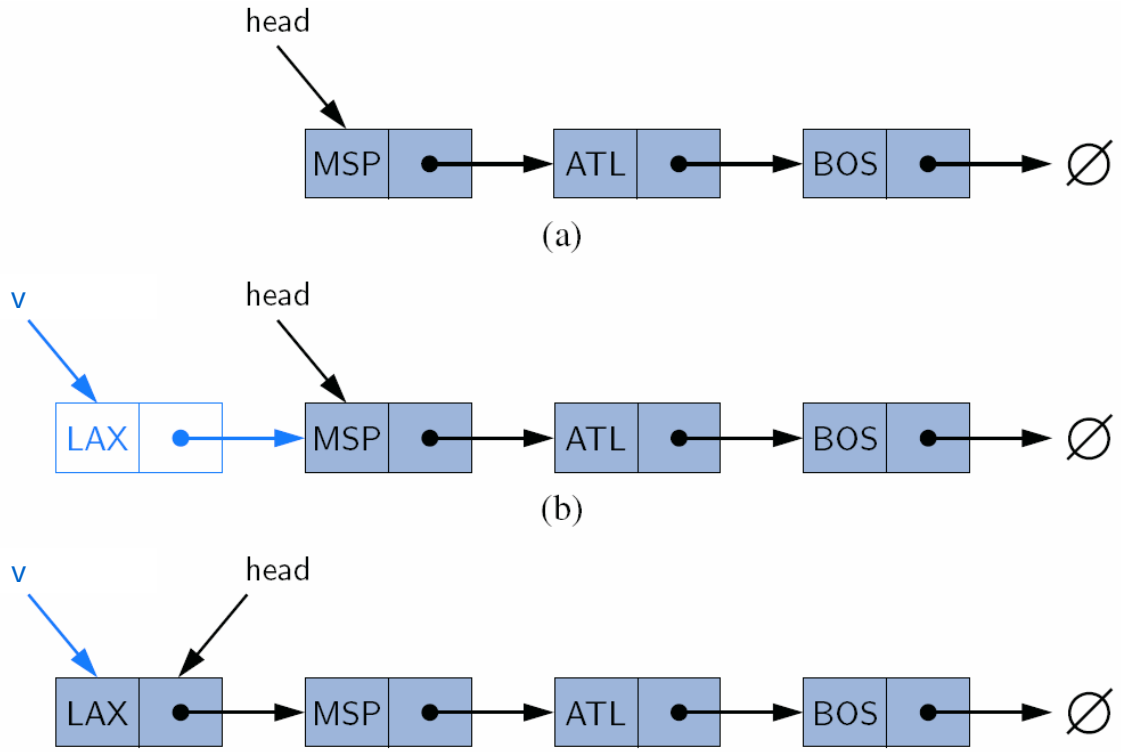


Kopf (und z.T. *Ende*) der Liste werden speziell gespeichert.

Den Kopf benötigt man als "Einstieg".

Singly-Linked-List: Einfügen eines Knotens

Einfügen am *Anfang (Kopf, head)* der Liste:



1. Verketten des neuen Knoten mit dem alten Kopf.

2. "head" updaten: Zeigt nun auf den neuen Knoten.

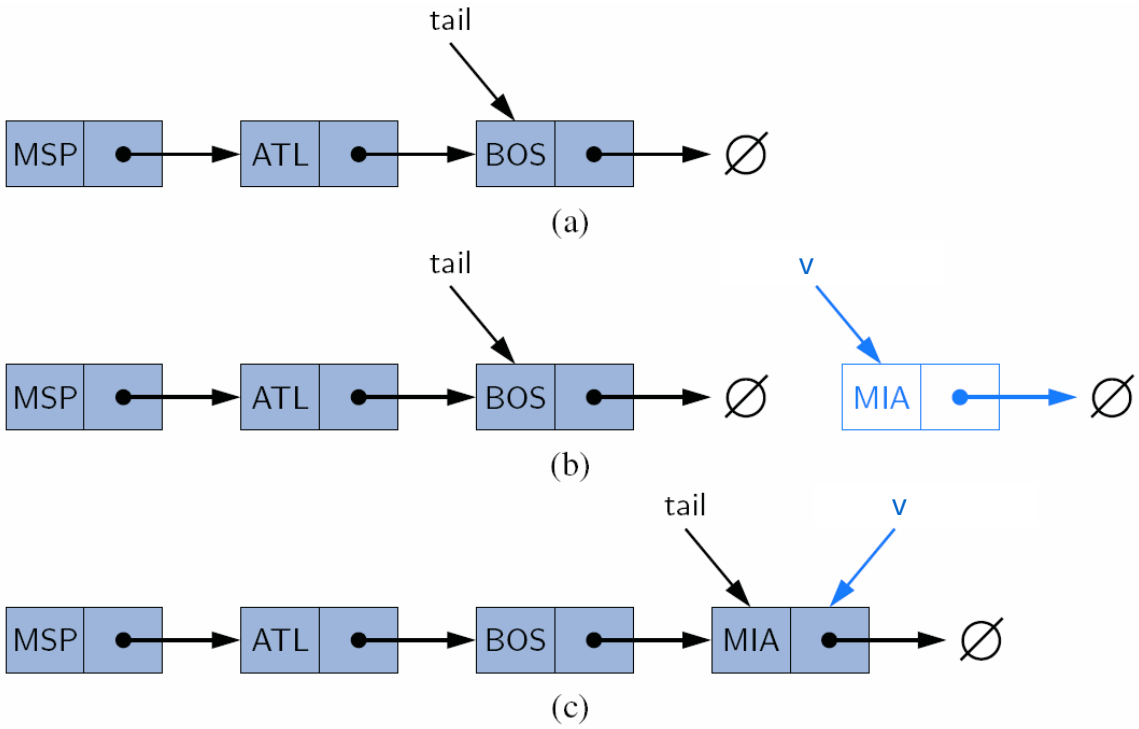
```
Algorithmus addFirst(v)
  v.setNext(head)        { Der neue Kopf verweist auf den alten }
  head ← v               { "head" zeigt nun auf den neue Knoten }
  size ← size + 1        { Anzahl Knoten hat um 1 zugenommen }
```

⇨ Laufzeit: *O(1)*

Singly-Linked-List: Einfügen eines Knotens

Einfügen am *Ende (tail)* der Liste:

1. Den neuen Knoten auf ***null*** zeigen lassen (Ende).
2. Verketten des früheren Endknoten mit dem neuen Knoten.
3. "tail" updaten: Zeigt nun auf den neuen Knoten.

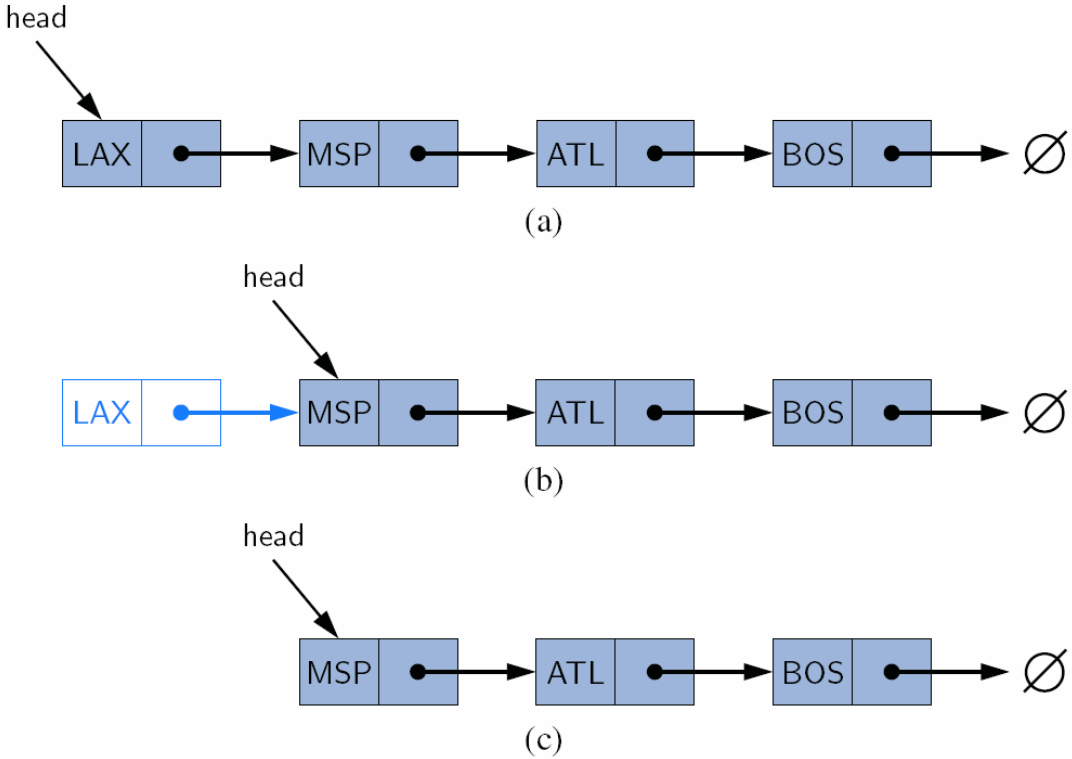



```
Algorithmus addLast(v)
  v.setNext(null)     { Neuer Knoten verweist auf "null " }
  tail.setNext(v)     { Alter "tail" verweist auf neuen "tail" }
  tail ← v            { "tail" zeigt nun auf den neue Knoten }
  size ← size + 1     { Anzahl Knoten hat um 1 zugenommen }
```

Singly-Linked-List: Entfernen eines Knotens

Entfernen am *Anfang (Kopf, head)* der Liste:

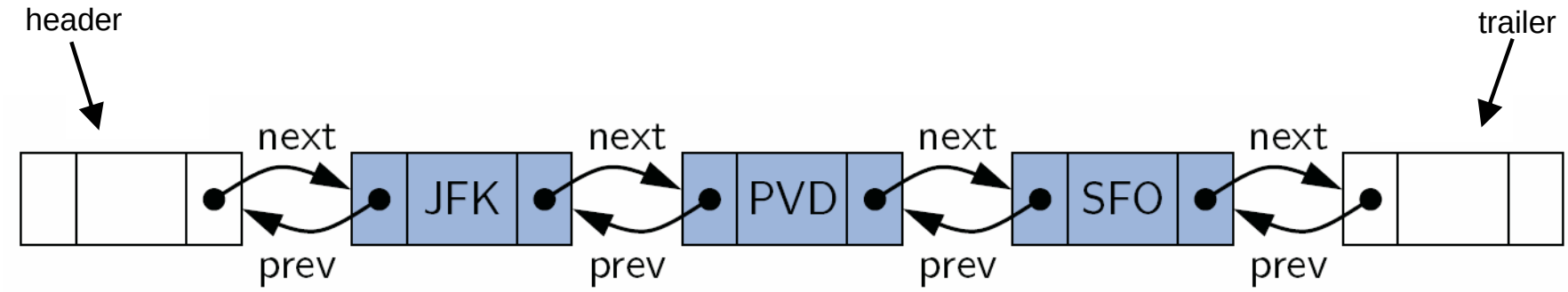
1. Überprüfung auf leere Liste.
2. "head" updaten: Zeigt nun auf den 2. Knoten.
3. Verbindung des alten "head"-Knotens mit der Liste löschen.




```
Algorithmus removeFirst()
  if (head == null) then  indicate an error: the list is empty
  t ← head                { Temporäre Referenz "t" auf alten "head" }
  head ← head.getNext()   { "head" zeigt neu auf 2. Knoten }
  t.setNext(null)         { Alten "head" aus Liste ausklinken }
  size ← size - 1         { Anzahl Knoten hat um 1 abgenommen }
```

Doppelt verkettete Liste (Doubly-Linked-List)

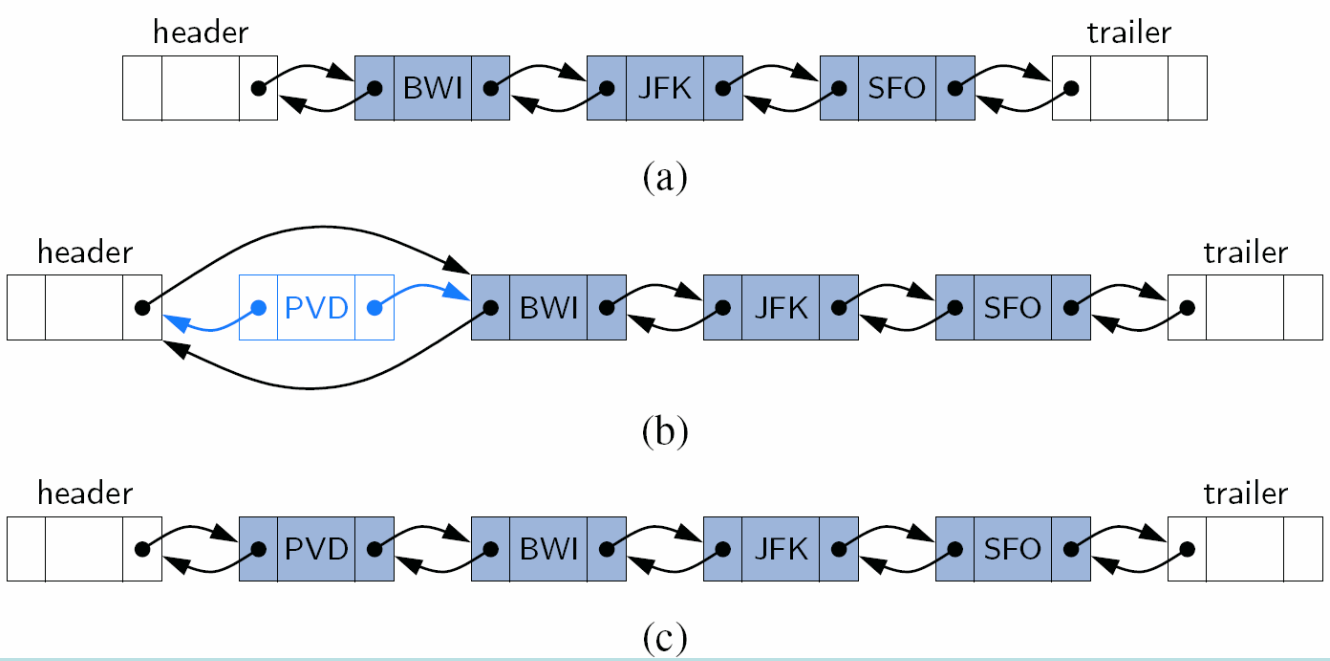
- *Singly-Linked-Lists* haben den Nachteil, dass man immer mit dem Kopf-Knoten beginnen muss und nur in eine Richtung suchen kann.
- Doubly-Linked-Lists beheben diesen Mangel auf Kosten des Speicherbedarfs:
  - Jeder Knoten speichert die Verbindung zum Vorgänger und zum Nachfolger.
  - Zudem werden häufig "header" und "trailer" spezielle Knoten sein, sogenannte *Sentinels* (oder *Guards*).
    Header und *Trailer* sind Start-Knoten für die Suche.



Doubly-Linked-List: Einfügen eines Knotens

Einfügen am *Anfang (Kopf, head)* der Liste:

```
Algorithmus addFirst(v)
  w  ← header.getNext()    { erster echter Listen-Knoten }

  v.setNext(w)
  v.setPrev(header)

  w.setPrev(v)
  header.setNext(v)

  size = size +1
```

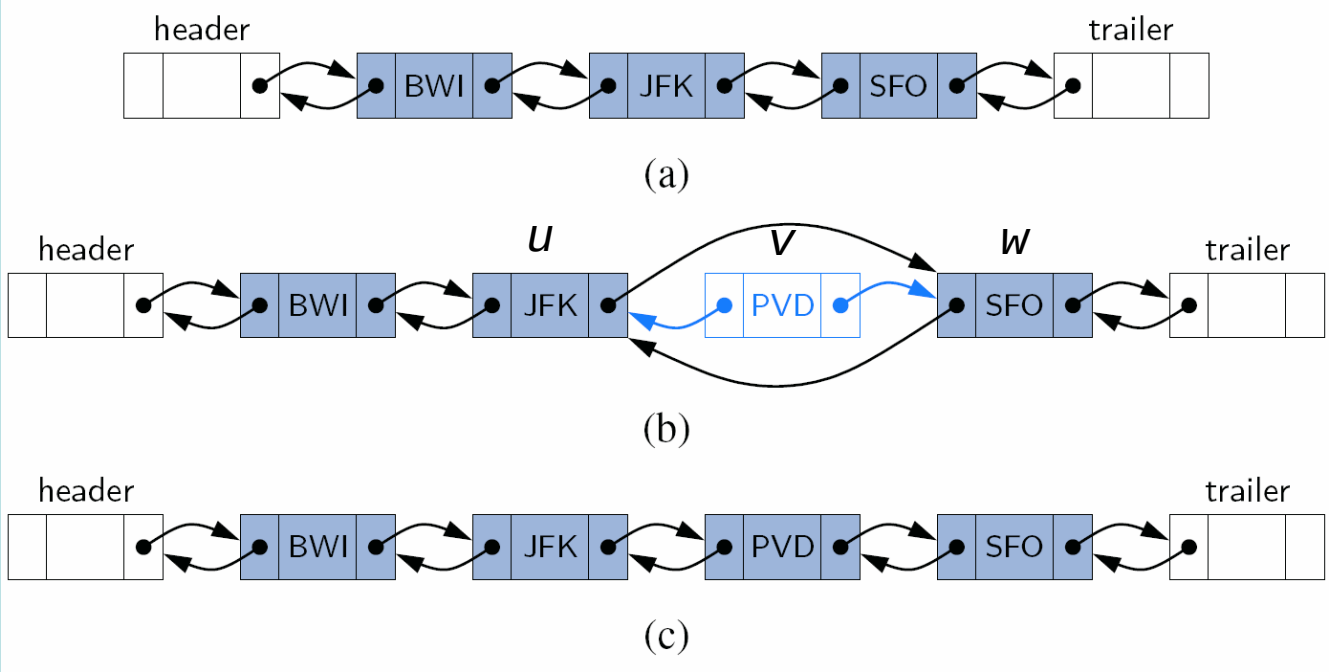


Doubly-Linked-List: Einfügen eines Knotens

Einfügen *nach einem bestimmten Knoten* in der Liste:

Algorithmus `addAfter(u, v)` *{ v nach u einfügen }*

```
w ← u.getNext()

v.setPrev(u)
v.setNext(w)

w.setPrev(v)
u.setNext(v)

size = size +1
```



⇨ Laufzeit: *O(1)*

Zusammenfassung
Array-List und Linked-List

Operation	Laufzeit	
	Array-List:	Linked-List:
`get(index)`	O(1)	O(n)
`set(index, data)`	O(1)	O(n)
`add(index, data)`	O(n)	O(n)
`addFirst(data)`	O(n)	O(1)
`addAfter(u, v)`	-	O(1)
`addLast(data)`	O(n)	O(1)
`remove(index)`	O(n)	O(n)

Beispiele

	Java	**Python**
Array-List:	*java.util.**ArrayList***	***list***
Linked-List:	*java.util.**LinkedList***	*collections.**deque*** (Zugriff nur auf Anfang und Ende der Liste. Siehe auch letzter Abschnitt 'Double-Ended Queues')

Iteratoren

- Ein *Iterator* abstrahiert den Prozess des schrittweisen *Scannens/Iterieren* über eine Sammlung von Elementen.
- Haupt-Operation eines *Iterators* :
  - Element **next**()
- Implementierung ist besonders einfach zu realisieren für Arrays oder einfach verkettete Listen.
- Ein Iterator ist typischerweise eng mit der zu iterierenen Datenstruktur verbunden.
- Details sind jeweils abhängig von der verwendeten Technologie.

Python

Erzeugung:

```
my_iterator = iter(iterable)
# oder:
my_iterator = iterable.__iter__()
```

Benutzung:

```
next_elment = next(my_iter)
# oder:
next_elment = my_iterator.__next__()
```

Ende der Iteration:

```
raise StopIteration
```

Java

Erzeugung:

```
Iterator<Type> myIterator =
   iterable.iterator()
```

Benutzung:

```
if (myIterator.hasNext()) {
  Type nextElement = myIterator.next()
```

Ende der Iteration:

```
myIterator.hasNext() == false
// resp.
throw new NoSuchElementException
```

Modifikation auf Iterable

Wichtig ist jeweils auch die Frage, ob eine *Iterable* modifiziert werden kann/darf, während ein *Iterator* darauf existiert:

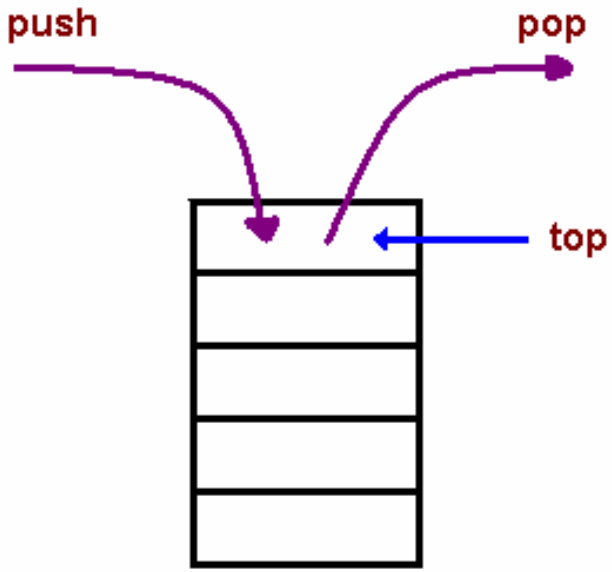
- *Python:* Ja
- *Java:* Nein: *ConcurrentModificationException*

Abstrakte Datentypen (ADTs)

- Ein ***abstrakter Datentyp*** (ADT) ist eine mögliche Abstraktion einer konkreten Datenstruktur
- Ein ADT spezifiziert:
  - Datenfelder / Attribute
  - Operationen / Methoden auf und mit den Attributen
  - Ausnahmen und Fehler der Methoden

- **Beispiel**: ADT Modellierung eines einfachen Börsensystems
  - Daten: Kauf/Verkauf-Aufträge
  - Operationen:
    - order **buy**(stock, shares, price)
    - order **sell**(stock, shares, price)
    - void **cancel**(order)
  - Error Conditions:
    - Kaufen/verkaufen von nicht-existierenden Aktien
    - Cancel nicht-existierender Aufträge.

Stack



- Der *Stack ADT* speichert beliebige Objekte.
- Einfügen und Entfernen erfolgt gemäss dem *last-in first-out* (LIFO) Schema
  - Wie ein Bonbonstift
  - Wie ein Tellerstapel
- Hauptoperationen des *Stacks* :
  - **push**(Object): ein Element *einfügen*.
  - Object **pop**(): *entfernen* und *zurückgeben* des obersten Elements.

- Zusätzliche Stack-Operationen:
  - Object **top()**: liefert das zuletzt eingefügte Element, *ohne* dass dieses entfernt wird.
  - integer **size()**: liefert die Anzahl gespeicherter Elemente
  - boolean **isEmpty()**: zeigt an, ob überhaupt Elemente gespeichert sind.

Stack Beispiel

Method	Return Value	Stack Contents
push(5)	–	(5)
push(3)	–	(5, 3)
size( )	2	(5, 3)
pop( )	3	(5)
isEmpty( )	false	(5)
pop( )	5	( )
isEmpty( )	true	( )
pop( )	null	( )
push(7)	–	(7)
push(9)	–	(7, 9)
top( )	9	(7, 9)
push(4)	–	(7, 9, 4)
size( )	3	(7, 9, 4)
pop( )	4	(7, 9)
push(6)	–	(7, 9, 6)
push(8)	–	(7, 9, 6, 8)
pop( )	8	(7, 9, 6)

Stacks-Anwendungen

Direkte Anwendungen

- History im Web Browser (besuchte Seiten)
- "Undo" Operationen im Editor
- Run-Configuration's in der Entwicklungsumgebung
- Methodenaufrufe in der Laufzeit-Umgebung
  - ⇨ Rekursion!

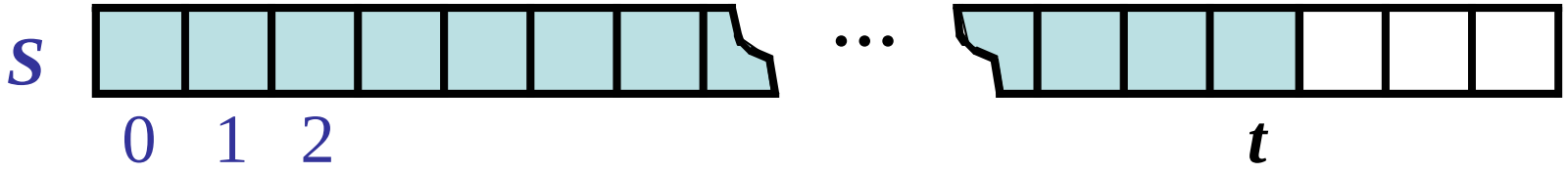
Indirekte Anwendungen

- Hilfsdatenstruktur für andere Algorithmen
- Stacks als Komponenten in anderen Datenstrukturen

Array-basierter Stack (1/3)

- Eine einfache Stack ADT Implementierung mit fixer Länge ist mithilfe eines Arrays möglich.
- Elemente werden von links nach rechts in das Array eingefügt.
- In einer Variable wird der Index des obersten Elements abgespeichert: ***t***

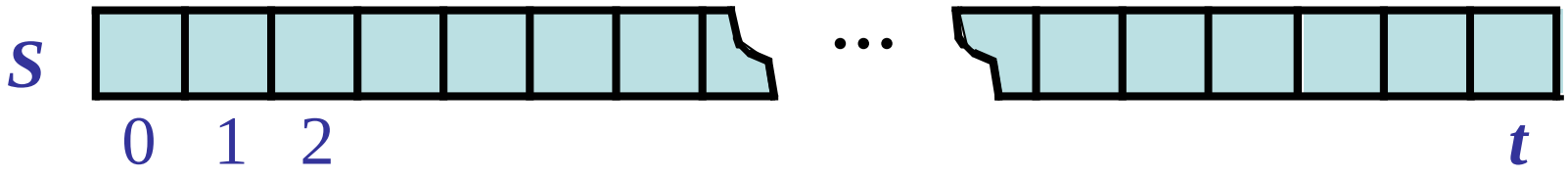
```
Algorithm size()
  return t + 1

Algorithm pop()
  if isEmpty() then
    return null
   else
    t ← t – 1
    return S[t + 1]
```



Array-basierter Stack (2/3)

- Das Array könnte voll sein, dann könnten keine weiteren Elemente mehr eingefügt werden.
- In diesem Fall würde die `push()` Operation eine `IllegalStateException` werfen:
  - Dies ist eine implementierungs-spezifische Einschränkung
  - Diese hat nichts mit dem Stack ADT zu tun.

```
Algorithm push(o)
  if t = S.length – 1 then
    throw IllegalStateException
   else
    t ← t + 1
    S[t] ← o
```



Array-basierter Stack: Performance und Limitationen (3/3)

Performance

- Falls sich n Elemente im Stack befinden, dann wird $O(n)$ Speicher benötigt.
- Jede Operation benötigt $O(1)$ Laufzeit

Limitationen

- Die maximale Grösse des Stacks wird mit der Konstruktion festgelegt und ist unveränderlich.
- Der Versuch ein weiteres Element in den vollen Stack einzufügen, führt zu einer implementierungs-spezifischen Exception.

Queue

- Der *Queue ADT* speichert beliebige Objekte.
- Einfügen und Entfernen erfolgt gemäss dem *first-in first-out* (FIFO) Schema.
- Das Einfügen erfolgt am Ende der Queue, das Entfernen am Anfang.

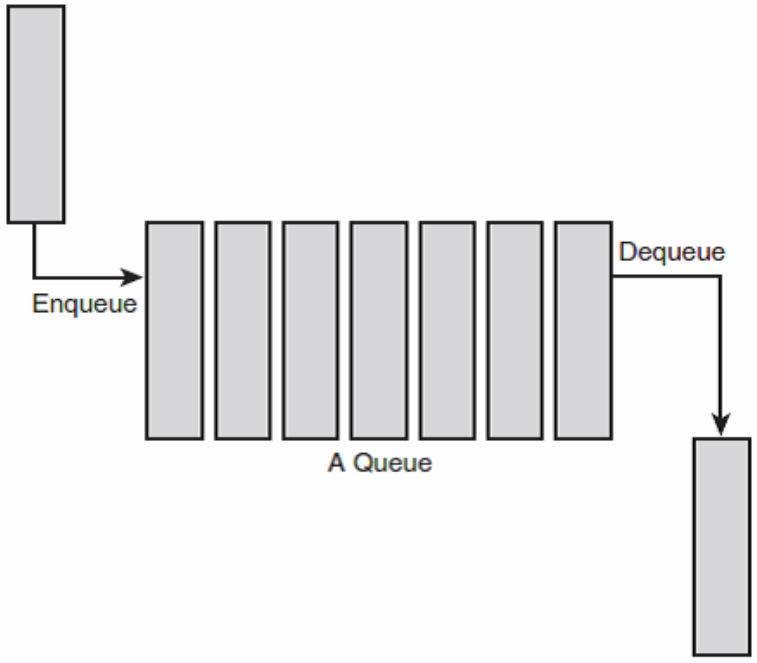
- Hauptoperationen der *Queue* :
  - **enqueue**(Object): einfügen eines Elementes am Ende der Queue.
  - Object **dequeue**(): entfernen und zurückgeben des Elements vom Anfang der Queue.

- Hilfsoperationen:
  - Object **first**(): liefert das erste Element, ohne es zu entfernen.
  - integer **size**(): liefert die Anzahl gespeicherter Elemente.
  - boolean **isEmpty**(): zeigt an, ob noch Elemente gespeichert sind.

- Randbedingungen:
  - Die Ausführung von **`dequeue()`** und **`first()`** auf einer leeren Queue bewirkt die Rückgabe von **`null`**.

Queue ADT - schematisch

- Hauptoperationen der Queue :
  - **enqueue**(Object): einfügen eines Elementes am Ende der Queue.
  - Object **dequeue**(): entfernen und zurückgeben des Elements vom Anfang der Queue.



Queue Beispiel

Operation	***Ausgabe***	*Q*
enqueue(5)	–	(5)
enqueue(3)	–	(5, 3)
dequeue()	*5*	(3)
enqueue(7)	–	(3, 7)
dequeue()	*3*	(7)
first()	*7*	(7)
dequeue()	*7*	()
dequeue()	*null*	()
isEmpty()	*true*	()
enqueue(9)	–	(9)
enqueue(7)	–	(9, 7)
size()	*2*	(9, 7)
enqueue(3)	–	(9, 7, 3)
enqueue(5)	–	(9, 7, 3, 5)
dequeue()	*9*	(7, 3, 5)

Anwendungen von Queues

- Direkte Applikationen
  - Wartelisten, Schalter, Zoll, Skilift
    (*First come, first served*)
  - Zugriff auf gemeinsame Ressourcen (z.B. Printer)
  - Multiprogramming, Multithreading

- Indirekte Applikationen
  - Als Hilfskonstrukte für andere Algorithmen
  - Als Komponenten für andere Datenstrukturen

Double-Ended Queues

Einfügen und Entfernen erfolgt am Anfang ***oder*** Ende der Queue:

Deque ADT

- Der *Double-Ended-Queue ADT* speichert beliebige Objekte. (*Double-Ended-Queue* ⇨ *DeQueue* ⇨ *Deque*)
- Einfügen und Entfernen erfolgt gemäss dem *first-in first-out* (FIFO) Schema.
- Das Einfügen und Entfernen erfolgt am Anfang **oder** Ende der Queue.

Hauptoperationen der *Deque* :

- **addFirst**(E):
  einfügen eines Elementes am Anfang der Queue.
- **addLast**(E):
  einfügen eines Elementes am Ende der Queue.
- E **removeFirst**():
  entferne das erste Element der Deque und gib es zurück.
- E **removeLast**():
  entferne das letzte Element der Deque und gib es zurück.
- E **first**():
  liefert das erste Element, ohne es zu entfernen.
- E **last**():
  liefert das letzte Element, ohne es zu entfernen.

Randbedingungen:

- Beim Auslesen eines Elements aus einer leeren Deque wird jeweils **`null`** zurückgegeben.

Deque Beispiel

Operation	Output	DQ
addFirst(3)	-	(3)
addFirst(5)	-	(5,3)
removeFirst()	5	(3)
addLast(7)		(3,7)
removeFirst()	3	(7)
removeLast()	7	()
removeFirst()	null	()
isEmpty()	true	()

Deque - Performance

Eine *Deque* realisiert mit einer *Doubly-Linked-List* :

Methode	O-Verhalten
size, isEmpty	O(1)
getFirst, getLast	O(1)
addFirst, addLast	O(1)
removeFirst, removeLast	O(1)